

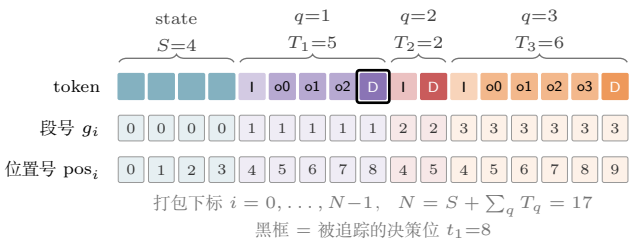
state 只编码一次，任意多个互相看不见的 question 分支共读它；每个分支末位直接读出长度等于自身选项数的概率分布，confidence 由该分布按算术公式算出，全程不逐 token 解码。

$$\mathbf{H} = f_{\theta}(\mathbf{X}; \mathbf{M}), \quad \mathbf{h}_q = \mathbf{H}[t_q, :], \quad \mathbf{p}_q = \text{softmax}(\mathbf{W}_{:K_q} \mathbf{h}_q + \mathbf{b}_{:K_q}), \quad p_q^{\text{noul}} = \sigma(\mathbf{w}^{\top} \mathbf{h}_q + b)$$

$$M_{ij} = \mathbf{1}[j \leq i] \cdot \mathbf{1}[g_j = 0 \vee g_j = g_i], \quad c_q = \frac{\max_k p_{q,k} - 1/K_q}{1 - 1/K_q}$$

索引： ① 打包 p.1 ② 树形 mask p.1 ③ 读出 p.1 ④ 事后算术 p.1 ⑤ 三种 mask p.2 ⑥ 位置号 p.2 ⑦ 精确验证 p.2 ⑧ Jev 位置实测 p.2 ⑨ 实测结论 p.3 ⑩ 接口与 $K_{\max}$ p.4 ⑪ 训练与数据 p.4

① 打包：一条序列装下 1 段 state 与 Q 个分支 〔token 计数可加：已实测；打包实现：推断〕



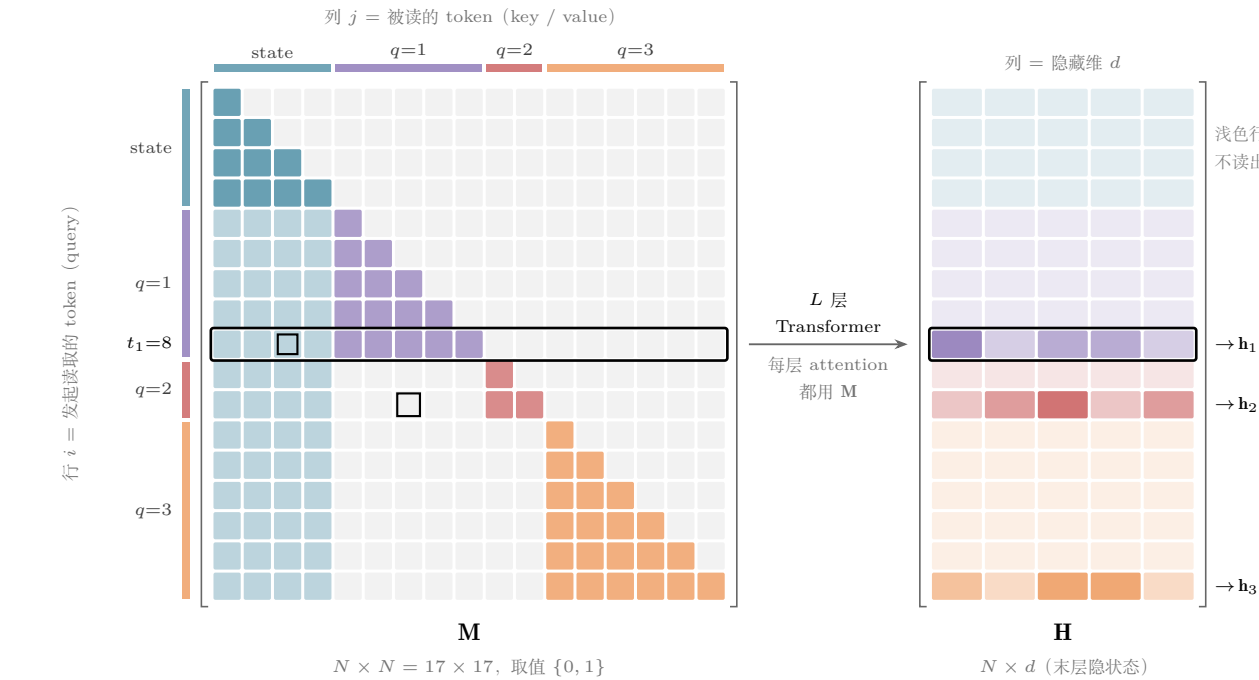
一格 = 1 个 token ($\mathbf{X}$ 的一行)。分支 = 指令 I + K_q 个选项 o + 决策位 D (示意中各压成 1 个 token)：Choice / Score 的 $T_q = K_q + 2$, Noul 没有选项, $T = 2$ 。图中 $q=1$ Choice ($K=3$)、 $q=2$ Noul、 $q=3$ Score ($K=4$) $\Rightarrow T = (5, 2, 6)$ 。

上限：整条序列 $N \leq 2^{16}$ ，其中 state 只计一次（已公开）；单个分支 $S + T_q \leq 2^{15}$ ，与各分支的位置号都从 S 续编一致（推断，见第 2 页）。

本次直连 Jev 实测 input_tokens：共享部分 271；每个 Noul+12；每多 1 个 Choice 选项或 1 个 Score 档 +6；Noul + Choice(3) + Score(3) 同发，预测 271+12+35+37 = 355，实测 355。question id 不计入输入 (283 $\rightarrow$ 283)，option key 计入 (300 $\rightarrow$ 310)。

- 变 Q 只是多拼几段，变 K_q 只是该段多几个选项 token：改变的是序列长度 N ，任何权重的形状都不变。

② 树形 mask：state 的 KV 复用，分支互相隔离 〔state 只读入一次：已公开；隔离：已实测；mask 本身：推断〕



state $\rightarrow$ **state**：每层只算一次，得到 $\mathbf{K}_{\ell}^S, \mathbf{V}_{\ell}^S$ ，形状 $S \times h_{kv} \times d_h$, $\ell = 1..L$ 。

分支 $\rightarrow$ **state**： Q 个分支读同一份 $\mathbf{K}^S, \mathbf{V}^S$ (复用)，state 不再重算。

分支内 causal：后面的选项看得见前面的选项，决策位 D 看得见整张选项表。

$M_{ij} = 0$ ：未来位置，或别的 question。

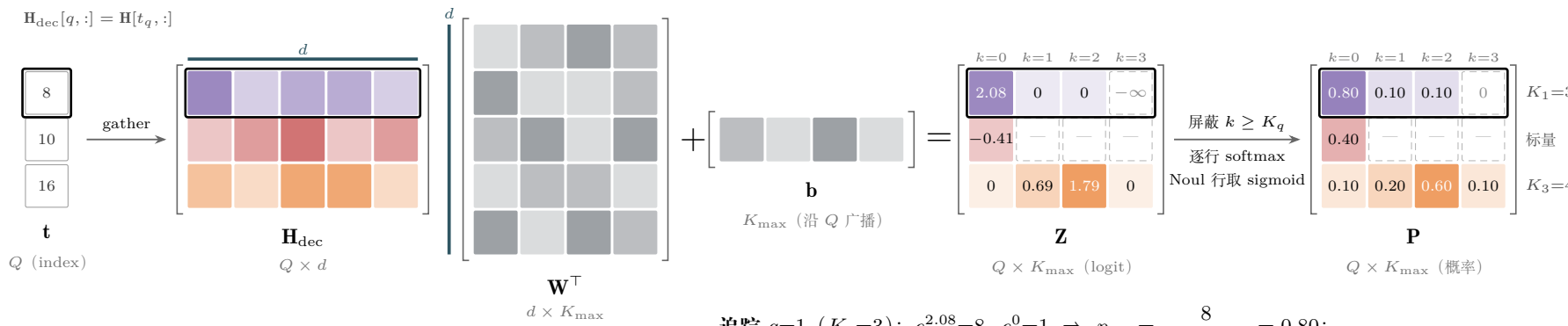
算两格（图中小黑框）：

$M_{8,2} = \mathbf{1}[2 \leq 8] \cdot \mathbf{1}[g_2=0] = 1$ ：
 $q=1$ 的 D 读 state 的第 2 个 token。

$M_{10,6} = \mathbf{1}[6 \leq 10] \cdot \mathbf{1}[g_6=0 \vee g_6=g_{10}]$
 $= 1 \cdot 0 = 0$ ： $g_6=1, g_{10}=2$,
 $q=2$ 读不到 $q=1$ 的 token。

- state 的每层 K/V 只算一次（深青）， Q 个分支各读这份缓存（浅青）加本分支自己的前缀；分支之间 $M = 0$ ：本次实测，暗号写在兄弟 question 里时 $p = 0.00$ (5/5 次)，写进 state 时 $p = 0.89\text{--}0.91$ 。 N 行末层隐状态里只读出 Q 个决策位。state 的处理量由 QS 降为 S ： $S=23\text{k}$ 、 $Q=5000$ 时逐题重算约 1.15×10^8 token，打包后不超过 65,536。缓存只在一次请求内复用，没有 decode 循环，请求结束即可释放。

③ 读出：决策位隐状态 $\rightarrow K_q$ 维概率 〔不逐 token 生成：已公开、已实测；槽位头或 pointer、Noul 标量头：推断〕



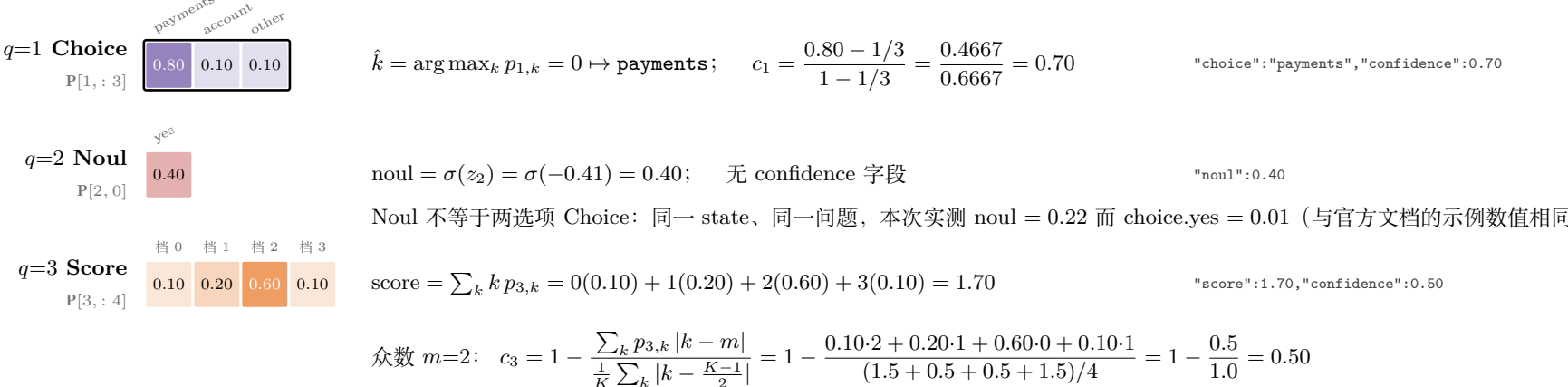
追踪 $q=1$ ($K_1=3$): $e^{2.08}=8, e^0=1 \Rightarrow p_{1,0} = \frac{8}{8+1+1} = 0.80$;

槽 $k=3$ 被屏蔽: $e^{-\infty}=0 \Rightarrow p_{1,3}=0$ 。 Noul 行: $p_2 = \sigma(-0.41) = \frac{1}{1+e^{0.41}} = 0.40$ 。

$\mathbf{Z}, \mathbf{P}$ 为示意数值；Jev 的 logit 不可观测。Noul 用独立标量读出属推断；「Noul $\neq$ 两选项 Choice」已实测，见 ④。

- 每个 question 只贡献 1 个向量 $\mathbf{h}_q$ ，一次矩阵乘得到全部槽位的 logit； $k \geq K_q$ 的槽位置 $-\infty$ 后逐行 softmax，Choice / Score 行的概率质量只落在自己的 K_q 个选项上、行和为 1。概率由一次前向直接读出，没有逐 token 解码：本次实测 $K=200$ 与 $K=2$ 的端到端耗时中位数为 316 与 309 ms； $Q=300$ 时 output_tokens = 5,594，而总耗时 376 ms。追加一个无关选项后 $\ln \frac{p(\text{customer})}{p(\text{unknown})}$ 由 -1.63 变为 -2.00 （各 5 个请求，区间不重叠）；若各选项 logit 独立且不变，该差值不会变，所以 $\mathbf{h}_q$ 读的是整张选项表。

④ 事后算术：choice / score / confidence 都在网络之外算 〔公式：官方 adapter 已公开；取全精度 p：已实测〕



c 依赖 K ：同一 $p_{\max} = 0.80$, $K=2 \Rightarrow c=0.60$, $K=3 \Rightarrow 0.70$, $K=5 \Rightarrow 0.75$ ； c 度量 $p_{\max}$ 高出均匀分布 $1/K$ 的比例。

复核：若 c 由响应里两位小数的 $\mathbf{p}$ 算出，重算值应 100% 吻合；文章证据包 1,657 条仅 36.8% 吻合， $K=2$ 时差值分布为 $\{-0.01:126, 0:254, +0.01:128\}$ ；本次直连实测 1,117 条， $\max |\text{差}| \leq 0.03$, $K=2$ 为 $\{-0.01:48, 0:144, +0.01:65\}$ ，与「先用全精度 $\mathbf{p}$ 算 c ，再各自舍入到两位」的预测一致 ($c = 2p - 1$ 使舍入误差加倍)。官方文档示例 $\mathbf{p} = (0.88, 0.12, 0.00)$ 、 $c = 0.81$ 同理：两位小数代入得 0.82。

output_tokens 也是事后量：Noul 为 $4 + \sum_q (15 + |\text{id}_q|)$ ，随不送入模型的 question id 变长 (21 $\rightarrow$ 32)，且与返回数值无关 (0.0 与 0.01 计数相同)。

- choice、score、confidence 都是对 $\mathbf{p}$ 的确定性算术，JSON 由应用代码序列化； c 不是第二个网络输出，也不是「答对概率」的另一个估计——分布集中而答案错误时， c 同样很高。

轴	S : state token 数； Q : question 数, $q \in \{1..Q\}$ ； K_q : 第 q 题选项数 (Choice ≤ 255 , Score ≤ 10 ; Noul 为 1 个标量), $k \in \{0..K_q-1\}$; T_q : 第 q 分支 token 数； $N = S + \sum_q T_q$: 打包长度, $i, j \in \{0..N-1\}$ 为 query / key 位置； d : 隐藏维； $K_{\max}$: 槽位预算（图中 4；接口上限 Choice 255、Score 10，见第 4 页 ⑩）； L : 层数； $h_{kv} \times d_h$: 每层 KV 头数 $\times$ 头维。
对象	$g_i \in \{0..Q\}$ 段号 (0 = state) 与 t_q 决策位下标：整数 index； $\mathbf{M} \in \{0, 1\}^{N \times N}$: mask； $\mathbf{Z}$: logit, 被屏蔽处为 $-\infty$ ； $\mathbf{P}$: 概率，Choice / Score 行非负、行和为 1、支撑集大小 K_q , Noul 行为单个 $p \in [0, 1]$ ； $c_q \in [0, 1]$: 由 $\mathbf{p}_q$ 算出的统计量，非学习输出。
机制	变 Q = 多拼几段 + mask 多几个对角块；变 K_q = 该段多几个选项 token + 读出时屏蔽多余槽位；两者都不改变权重形状。另一种同样符合证据的读出是 pointer: $z_{q,k} = \langle \mathbf{h}_q, \mathbf{U}\mathbf{o}_{q,k} \rangle$, $\mathbf{o}_{q,k}$ 为选项 k 末 token 的隐状态，不需要槽位上限。
一格	① 的一格 = 1 个 token；M 的一格 = 一对 (query i , key j) 是否可见； $\mathbf{Z}, \mathbf{P}$ 的一格 = 一个标量； $\mathbf{H}, \mathbf{H}_{\text{dec}}, \mathbf{W}, \mathbf{b}$ 的一格 = 一个分量（明暗为示意）。tile 数量表示轴结构，不是真实维度。

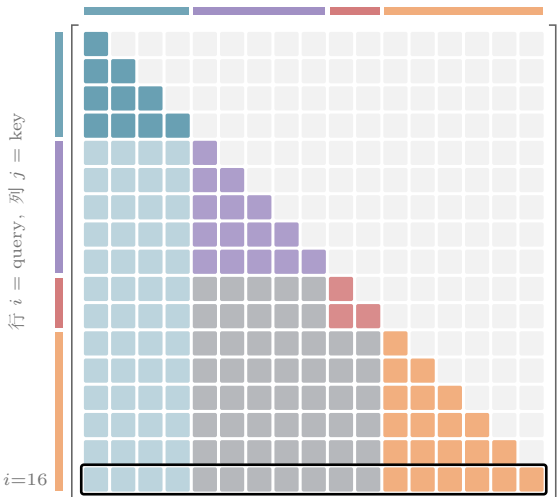
mask 决定「谁能读谁」，位置号决定「隔多远计分」，是 attention 的两个独立输入；只有位置号取树深度（每个分支都从 state 末尾续编），打包计算才与逐题单独运行逐位相等。

$$s_{ij} = \frac{1}{\sqrt{d_h}} (\mathbf{R}_{\text{pos}_i} \mathbf{q}_i)^\top (\mathbf{R}_{\text{pos}_j} \mathbf{k}_j) + \log M_{ij} = \frac{1}{\sqrt{d_h}} \mathbf{q}_i^\top \mathbf{R}_{\text{pos}_j - \text{pos}_i} \mathbf{k}_j + \log M_{ij}$$

$$M_{ij} = \mathbf{1}[j \in \text{anc}(i) \cup \{i\}], \quad \text{pos}_i = |\text{anc}(i)|$$

⑤ 同一条打包序列、同一组权重，只换 mask [背景知识；Jev 的行为与 (c) 一致：已实测]

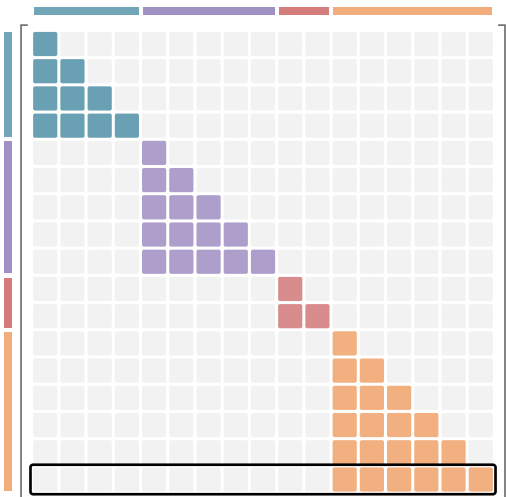
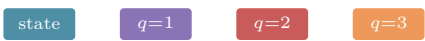
(a) 链: 普通 causal



$$M_{ij} = \mathbf{1}[j \leq i]$$

灰格 = 后面的 question 读得到前面 question 的全部 token：答案互相影响，并依赖提问顺序。

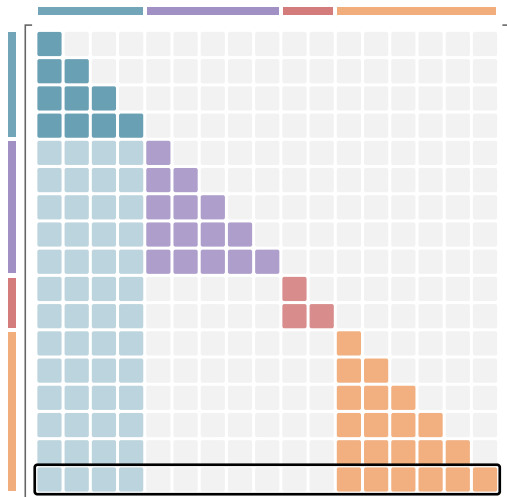
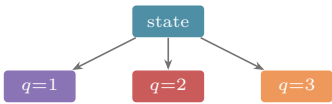
(b) 森林：按段隔离



$$M_{ij} = \mathbf{1}[j \leq i] \mathbf{1}[g_j = g_i]$$

分支互相隔离，但浅青区为空：question 读不到 state。训练时把多篇文档打包进一条序列用的就是它。

(c) 树：共享主干，隔离分支

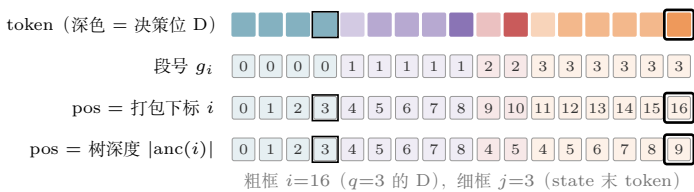


$$M_{ij} = \mathbf{1}[j \leq i] \mathbf{1}[g_j = 0 \vee g_j = g_i]$$

分支共读 state (浅青), 彼此之间为空。token i 的祖先 $\text{anc}(i) = \text{state}$ 全部 token + 本分支更早的 token。

- 黑框是 $q=3$ 的决策位那一行 ($i=16$): 链里它读得到 $q=1$ 、 $q=2$ 的全部 token (灰格), 森林里它读不到 state, 树里它读 state 加本分支。tree attention 最早用于投机解码里并行验证一棵候选 token 树 (SpecInfer, Miao et al. 2023; Medusa, Cai et al. 2024); 同一结构用于共享前缀的高吞吐推理见 Hydragen (Juravsky et al. 2024) 与 DeFT (Yao et al. 2024)。Jev 的树只有两层: 一条主干 (state) 加 Q 个叶分支。

⑥ 同一张树形 mask 可以配两种位置号 [数学事实]



- 位置号不属于 mask: mask 挡住了兄弟分文的**内容**, 挡不住它们的**长度**经由位置号进入旋转 $\mathbf{R}_\Delta$ 。取树深度时, 每个分支看到的 (内容, 相对距离) 与单独运行逐项相同; 取打包下标时, 分支 q 到 state 的距离被平移 $\sum_{r < q} T_r$, 结果随兄弟分支的长度与顺序改变。ALiBi (偏置 $\propto \text{pos}_i - \text{pos}_j$) 与可学习的绝对位置同理。

⑦ 精确验证：2 层 RoPE Transformer, 随机权重 (numpy) [已验证: repro/tree_mask_position_demo.py]

	h_1	h_2	h_3
树深度, 顺序 (1, 2, 3)	0	0	0
树深度, 顺序 (3, 2, 1)	0	0	0
打包下标, 顺序 (1, 2, 3)	0	1.2	1.6
打包下标, 顺序 (3, 2, 1)	1.8	0.34	0

$\max |h_q^{\text{打包}} - h_q^{\text{单独}}|$
 $4 \times Q; \text{虚线格} = 0 \ (\leq 2.9 \times 10^{-15}, \text{浮点舍入})$

- 所以「一条打包序列 + 树形 mask + 树深度位置号」与「先算一次 state 的 KV，再让 Q 个分支各自从位置 S 续算」是同一计算的两种实现，输出逐位相同，从 API 外部无法区分；省下的是 $(Q-1) \cdot S(S+1)/2$ 次 state 自注意，分支读 state 的 $\sum_q T_q S$ 仍然要算。

⑧ Jev 实测：分支输出不依赖兄弟分支的数量、长度、顺序 [已实测：各 8 次重复；位置号取法本身：推断]

同一 state、同一目标	question	input_tokens	Choice $p(\text{payments})$	Noul
目标单独发送		370	0.290 ± 0.019	0.260 ± 0.005
目标排在 400 个兄弟分支之前		21,354	0.296 ± 0.018	0.256 ± 0.007
目标排在 400 个兄弟分支之后		21,354	0.290 ± 0.017	0.258 ± 0.007
31 题版本 (单独 / 首 / 中 / 末)		354 / 1,265	0.281 / 0.281 / 0.288 / 0.289	—

均值 $\pm$ 单次重复的标准差。若位置号取打包下标,「之后」条件下目标分支到 state 的相对距离要多出约 20,984; 实测均值差 ≤ 0.006 , 小于单次重复的标准差。另一处吻合: 单分支上限 2^{15} 与整请求上限 2^{16} 并存, 只有位置号按分支才说得通。

- 结论：兄弟分支既不泄露内容（第 1 页 @: $p = 0.00$ ），也不改变目标分支看到的位置；这与「树深度位置号」或等价的「前缀缓存 + 分支从 S 续编」一致。同一请求重复发送时 p 本身有 ± 0.02 左右的波动，所以这里比较的是均值。

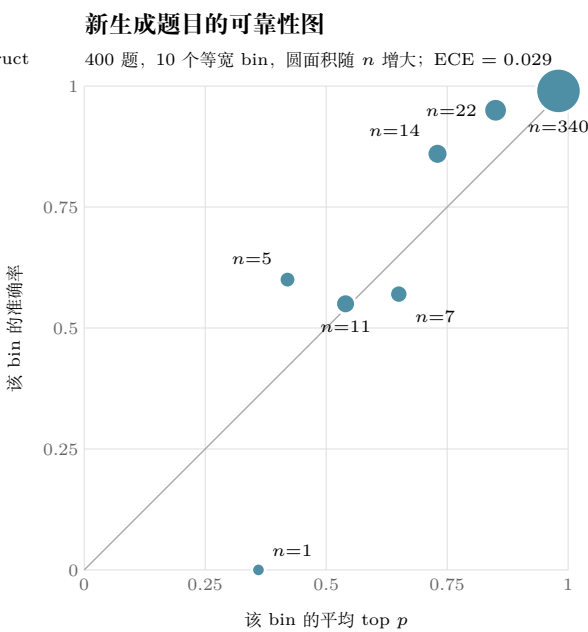
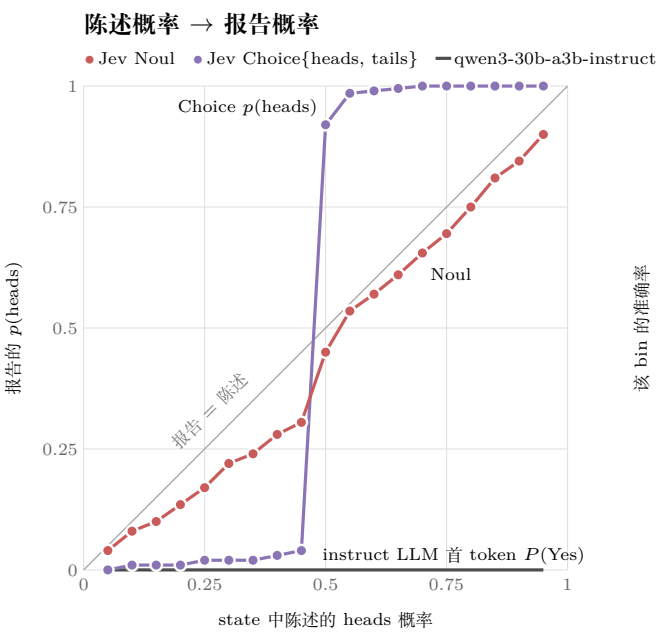
对象	$\text{anc}(i)$: token i 在 token 树上的祖先集合 (index 集合); pos_i : 整数位置号; $\mathbf{R}_\Delta$: RoPE 旋转, 只依赖相对距离 $\Delta = \text{pos}_j - \text{pos}_i$; ℓ : 一段 key 上的 logsumexp; $\mathbf{A}_S, \mathbf{A}_q$: 只在 state 段、只在本分支段上做的局部注意力输出。
机制	逐层归纳: 第 ℓ 层 token i 的输出只由 $\{j: M_{ij} = 1\}$ 的上一层隐状态与 Δ_{ij} 决定; 树深度编号下两者都与单独运行逐项相同, 所以每一层都相同。
边界	树形 mask 不提供跨 question 的一致性: 本次实测, 同一问题与其否定的两个 Noul 之和在 0.86–1.59 之间。后一问依赖前一问的答案时, 只能拆成两次请求或合并成一个 question。
一格	mask 的一格 = 一对 (query i , key j) 是否可见, 灰格 = 跨分支可见; 位置号一格 = 一个整数; ⑦ 的一格 = 一个误差标量。

对 Jev 1.13 的 250 次直连请求（10,249 个 question）复现了文章的全部关键行为，并新增两条：Noul 报告事件概率而 Choice 报告「哪个选项最可能」；概率在新生成、由代码判分的题目上是校准的。

$$\text{ECE} = \sum_b \frac{n_b}{n} |\text{acc}_b - \text{conf}_b|, \quad \arg \min_{\mathbf{p}} \mathbb{E}_{y \sim \pi} [-\log p_y] = \boldsymbol{\pi}$$

⑨ 实测确定性结论 [typesafe/jev-1.13, 经 OpenRouter /api/alpha/decisions, 2026-09-21; 脚本与原始记录: repro/jev_probes.py, .jsonl]

结论	实验设置	实测数值	与文章 / 官方对照
1	input_tokens 严格可加; question id 不进入模型, option key 进入	共享 271, Noul +12, 每选项、每档 +6; 预测 355 = 实测 355; id 283 → 283, key 300 → 310	文章 260 / +8 / +6, 318 = 318: 一致
2	question 之间没有信息通路	暗号放兄弟 question / 不放 / 放 state, 各 5 次	文章 0.00 / 0.00 / 0.90–0.92: 复现
3	分支输出与兄弟分支的数量、长度、顺序无关	目标单独 / 排在 400 个兄弟分支 (2.1 万 token) 之前 / 之后, 各 8 次	文章未测: 新增 (第 2 页)
4	读出读的是整张选项表, IIA 不成立	4 选项与追加无关第 5 项; 每请求 40 个重复题, 各 5 个请求	文章 +0.38 → +0.11: 复现 (场景不同)
5	决策能用排在最后的选项信息, 且对顺序敏感	参考卡放在选项首 / 中 / 末, 24 个请求	文章 12/16、11/16、16/16, 末位 0.88: 复现
6	没有逐 token 解码	K=2 与 200; Q=1 与 300; 各 6 次顺序请求	文章同结论: 复现
7	confidence 是全精度 p 的算术函数, 之后才舍入	本次全部 1,117 个 Choice / Score 答案	文章证据包 1,657 条: 同结论
8	Noul 不等于两选项 Choice; Noul 之间没有一致性约束	同一问题的 Noul / yes–no Choice / 否定 Noul, 10 个 state	官方示例 0.22 与 0.01: 数值相同
9	Noul 近似报告事件概率; Choice 报告哪个选项最可能	state 陈述 heads 概率 5%–95%, 19 档各 2 次; 公平骰子 3 次	新增 (下左图)
10	在新生成、代码判分的题目上是校准的	4 类各 100 题, 50 题一个请求	文章 MMLU ECE 0.031; 乘法 86.7% / 0.83, 本次 88% / 0.83
11	API 输出不是逐位确定的	同一请求 8 次; 同一请求内 40 个重复题	文章同: 复现
12	K 的上限在请求校验处强制: Choice 1–255, Score 1–10	Choice K=0, 1, 255, 256; Score K=1, 10, 11; type 取 text	官方文档: 255 与 10; 一致 (第 4 页)
13	选项的槽位编号不写成文字; 无信息时读出偏向第 0 个槽位	3 位数字 key, K=2.255 共 13 档; 全部选项相同的请求, K=10, 50, 255 各 3 次	新增 (第 4 页)



题型 (各 100 题)	准确率	top p
三位数 × 两位数 (4 选 1)	0.88	0.83
小数比大小	1.00	0.97
日期先后 (混合格式)	1.00	0.98
单词含某字母 (Noul)	0.96	0.95

题目由带种子的随机数现场生成、由代码判分，不可能出现在任何训练集里。模型最弱的一类（乘法）置信度也最低，中间几个 bin 略偏保守。左图的含义：同一个 state，Noul 给出的是「这件事发生的概率」，Choice 给出的是「这个选项是最佳答案的概率」；两者都叫 probability，但不是同一个量。

● Choice 的 p 与 confidence 描述的是「选哪个」的把握，不是结果的发生概率：陈述 55% heads 时 Choice 给 heads 0.985、c = 0.97。需要事件概率时用 Noul；需要在互斥选项里挑一个时用 Choice；两者的阈值不能互相套用（官方 jaggedness 页有同样的提醒）。

对象	acc _b , conf _b : 第 b 个 bin 内的准确率与平均 top 概率; n _b : bin 内题数; π: 标签的真实条件分布; y: one-hot 标签; top p: Choice 取 max _k p _k , Noul 取 max(p, 1 − p)。
范围	全部数值针对 jev-1.13 (OpenRouter 永久标识 typesafe/jev-1.13-20260917), 单账号、单日、未控制服务端负载; 耗时含 OpenRouter 的转发开销; 每个条件的重复次数见表。
边界	「确定」指可由脚本重复得到的行为; mask、位置号、读出头、损失函数的具体形式仍属推断, 它们只是与这些行为相容的最简实现。

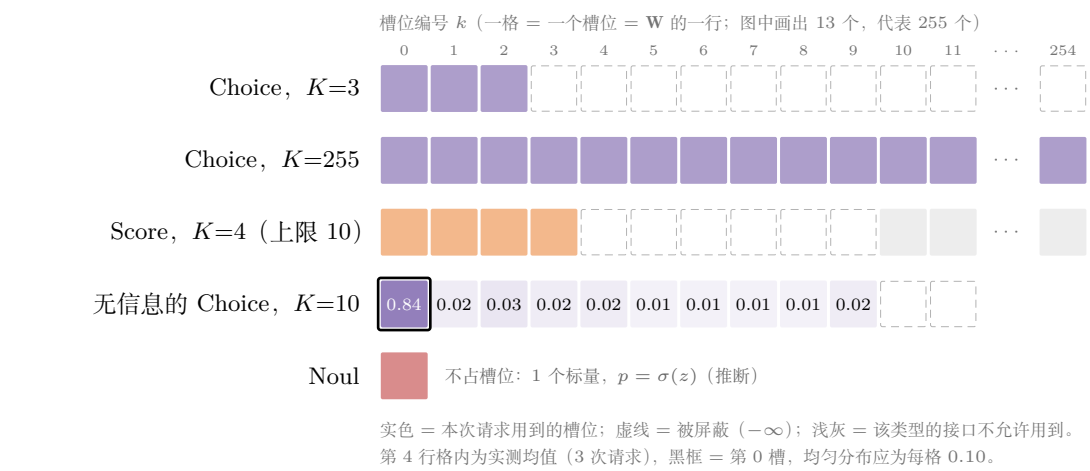
三种 question 类型对应三种固定大小的读出：Noul 是 1 个标量，Choice 是至多 255 个槽位上的 softmax，Score 是至多 10 个有序槽位； $K_{\max}$ 是训练时定下的槽位预算，请求里的 K_q 只决定用掉前几个槽位。

$$\mathbf{p}_q = \text{softmax}(\mathbf{z}_q + \mathbf{m}_q), \quad m_{q,k} = \begin{cases} 0, & k < K_q \\ -\infty, & k \geq K_q \end{cases} \quad 1 \leq K_q \leq K_{\max}$$

$$K_{\max}^{\text{choice}} = 255 = 2^8 - 1, \quad K_{\max}^{\text{score}} = 10, \quad \text{noul}: p = \sigma(z)$$

⑩ 接口约定与槽位预算 $K_{\max}$ 〔接口与上限：官方文档 + 已实测；读出头形式：推断〕

类型	请求里的 criteria	K 的范围（本次实测）	返回字段	读出（推断）
Noul	可选的 true / false 两段描述	没有选项，不占 K	noul $\in [0, 1]$ ；没有 confidence	1 个标量 logit，取 sigmoid
Choice	有序 map：option key $\rightarrow$ 描述或 null	$1 \leq K \leq 255$ ； $K=256$ 与 $K=0$ 返回 HTTP 400； $K=1$ 时 $p = 1$ 、 $c = 1$	choice (argmax 的 key)、 probabilities (K 项，和为 1)、 confidence	前 K 个槽位上的 softmax；槽位 $k =$ 列表第 k 项
Score	有序数组：第 k 项是第 k 档的描述	$1 \leq K \leq 10$ ； $K=11$ 返回 HTTP 400； $K=1$ 时 score = 0、 $c = 1$	score $= \sum_k k p_k$ 、 legend 、 probabilities 、 confidence （众数距离式）	前 K 个有序槽位上的 softmax，再取期望
其他	type 取别的值，例如 text	请求被拒：type 只能是以上三者	—	不允许字符串等序列输出，全部输出才能并行读出

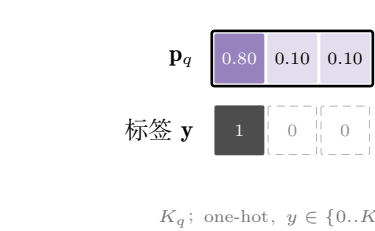


• K_q 可变而权重形状不变，靠的是「固定槽位预算 + 屏蔽」：读出头按 $K_{\max}$ 训练，请求只占用前 K_q 个槽位。没有信息可依时，读出并不回到均匀分布：全部选项相同的请求里第 0 槽得到 0.84 ($K=10$)、0.86 ($K=50$)、0.74 ($K=255$)，confidence 为 0.72–0.86；「低信息」不会自动表现为「低 confidence」，需要显式提供「无法判断」选项。同样三档内容，问成 Score 与问成 Choice 得到的分布不同 (10 个 state 上逐项最大差平均 0.075、最大 0.28，其中 1 例 argmax 翻转)：类型本身是条件的一部分，三种类型不共享同一个条件分布。

⑪ 训练与数据：公开了什么，没公开什么，行为给出了哪些约束 〔文章未实测训练；以下为逐条核对过的一手来源 + 本次行为实测〕

已公开 (TypeSafe 博客、FAQ、文档；Almeida 在 X 与 HN 的发言)

- RLCD 是继 RLHF、RLVR 之后对预训练语言模型的第三种后训练路径；概率针对 outcomes 优化以反映不确定性，校准按预测分组衡量。
- 「新的模型架构 + 并行采样器」；不允许字符串等序列型输出，因此全部输出可以并行计算。
- 训练数据 100% 为自制的合成数据，且不是 LLM 直接吐出的文本；自称数据研究实验室，研究投入绝大部分在做「通用」的数据上；认为新任务的瓶颈是数据而非架构，要避免能力参差需要极其多样的输入分布。
- 英语是主要训练语言，CJK 等精度较低。不用客户数据训练，所有账号共用同一份权重，没有按账号的微调或 LoRA。
- 自我界定：结构化数据模型，因不生成语言而「技术上不算语言模型」；「既不小，也不是 LLM」。架构暂不公开，团队讨论过写论文。
- 不发布公开基准。自家 workflow eval 以 GPT-6 Astra 与 Fable 5.1 的平均预测作参考概率，并称这些 workflow 不在训练分布内。
- 官方自述的弱项：字面化理解、计数与数值、日期比较、多跳间接、大而无关的 state、对抗性内容、Score 档位的数值校准、Noul 与 Choice 之间没有结构不变性。



算一个损失： $-\ln p_{q,y} = -\ln 0.80 = 0.223$ ；若正确答案是第 1 项则为 $-\ln 0.10 = 2.303$ 。为什么 proper：真实分布 $\pi = (0.7, 0.3)$ 时，报告 0.7 的期望损失是 0.611，报告 0.9 是 0.764，报告 0.5 是 0.693，如实报告最小。

与架构的对应 (推断)：(1) 损失直接作用在决策位读出上， $\mathbf{p}_q$ 对 θ 可微，不需要采样文本；以 $\log p_y$ 为奖励、以报告的分布为动作时，期望奖励的梯度与监督梯度相同。(2) 一条打包序列带 Q 个有标签的 question，一次前向得到 Q 个损失项，state 的计算在 Q 个样本间摊销。(3) 树形 mask + 树深度位置号使训练打包与推理逐位一致，question 的数量与顺序不构成分布偏移。(4) 槽位读出要对选项顺序不敏感，需要在数据里打乱选项顺序；实测仍有明显的首位偏置，说明这一点没有做到位。

• 关于训练，能确定的只有目标的形态 (对限定选项给出校准概率、不生成文本)、数据全部自制且为合成数据、不用客户数据；基座、标签来源与损失均未公开。行为上可以确定：三种类型学到的是三种语义不同的概率，校准在无法被记忆的可判分新题上成立。相关公开研究 (据检索)：预训练 LM 在选择题上本身校准良好、RLHF 之后变差 (Kadavath et al. 2022; GPT-4 Technical Report 2023)；以 proper scoring rule 作 RL 奖励可同时得到准确与校准 (Damani et al. 2025; Rewarding Doubt 2025)。

轴	K_q ：本题用到的槽位数； $K_{\max}$ ：读出头的槽位预算，随类型不同 (Choice 255, Score 10)； $k \in \{0..K_q-1\}$ ：槽位编号，等于选项在 criteria 里的次序，Score 的 k 同时是档位的数值。
对象	$\mathbf{m}_q$ ：屏蔽向量 (mask，取值 0 或 $-\infty$)； $\mathbf{y}$ ：one-hot 标签； π ：标签的真实条件分布；option key 是给应用代码用的名字，由应用代码把槽位 k 映射回 key。
机制	序列型输出 (字符串、列表) 被接口排除，所以每个输出都是固定大小的类别分布或标量，可以从单个决策位一次读出；候选数超过预算时 in 应用层拆成「独立打分 + 显式选择」两阶段。
一格	槽位图的一格 = 一个槽位 (W 的一行)；第 4 行格内 = 实测概率； $\mathbf{p}_q$ 、 $\mathbf{y}$ 的一格 = 一个标量。